

通訊序列設計 期中 PROJECT

核心問題 如何找到PCP

范榆君

陳品維

甘承翰

TABLE OF CONTENTS

- 固定配對窮舉法($L=50$)
- SDS建構法($L=74$ 、 82)
- 序列壓縮法($L=90$)

固定配對窮舉法 (EX. L=50)

- **固定 A 搜尋 B**，搭配存在於PCP的一個已知的 A 序列
 $\{ \text{---++--+--+---+--+---+--+---+--+---+--+---+--+---+--+---+} \}$ 且
 序列元素和 $a = 6$ 。
- **平方和定理**: $a^2 + b^2 = 2L = 100$
- **參數設定**: 已知固定序列 $a = 6$ ，則 $|b|$ 必須為 8。
- **權重 (Weight) 計算**: 如果讓 B 序列和為 8，長度 50 的序列 B 必須恰好包含 21 個 -1
- **效益**: 將搜尋空間從 2^{50} 大幅縮減至 $C(50, 21)$ 。

程式執行流程:



核心演算法：GOSPER'S HACK

用於搜尋空間生成 (Generation)

用於高效生成下一個「具有相同位元數 (Same Bit Count)」的組合。

- 這是一種**位元操作 (Bitwise)** 的技巧。
- **功能：** 給定一個整數 k (ex. $2^{22} - 1$)，它能高效地計算出**下一個比 k 大**，且讓二進位 **1 的個數與對應序列中-1個數達到固定數目**。
- **效率：** 運算只需常數時間 $O(1)$ ，不需迴圈

程式碼實作

```
// Gosper's Hack 核心邏輯
unsigned __int128 c = k_b & -k_b;
unsigned __int128 r = k_b + c;
k_b = (((k_b ^ r) >> 2) / c) | r;
```

Gosper's Hack 讓我們能直接跳躍生成所有

"含有 21 個 1 的 50-bit 整數"

完全跳過不符合權重 90% 以上的組合。

核心演算法：GOSPER'S HACK

用於搜尋空間生成 (Generation)

用於高效生成下一個「具有相同位元數 (Same Bit Count)」的組合。

Ex. B序列中1的個數固定4

Sequence	8-bits	
+Kb	01011100	← 第一組
-Kb	10100100	
c = Kb & -Kb+1	00000100	
r = Kb + c	01100000	
r' = r xor Kb	00111100	
r''=r' shift 2	00001111	
r'' / c	00000011	
(r'' / c) r	01100011	← 第二組

程式碼實作

```
// Gosper's Hack 核心邏輯
unsigned __int128 c = k_b & -k_b;
unsigned __int128 r = k_b + c;
k_b = (((k_b ^ r) >> 2) / c) | r;
```

Gosper's Hack 讓我們能直接跳躍生成所有
"含有 21 個 1 的 50-bit 整數"
完全跳過不符合權重 90% 以上的組合。

效能優化 (PRUNING & FILTERING)

- **預計算 A**: 只需計算一次 A 的 PAF。
- **剪枝原理**: 若 $PAF_A + PAF_B = 0$, 則 $|PAF_B|$ 絕不可能大於 $|PAF_A|$ 。

```
// 絕對值剪枝 (Pruning) if (abs(pacf_b_u) >  
max_pacf_a_abs) { return 0; // 提早淘汰 }
```

- **同構過濾**: 若序列反轉, 則跳過 (只保留標準型)。

```
//序列反轉  
If (reverse_int(k_b) < k_b){  
    skip= 1; //跳出迴圈  
}
```

UNIQUE PCP FOR L=50

Pair #1 PAPR: 4.4229 (6.457 dB)

SEQ A (PAPR=3.5241): { $-\quad-\quad-\quad-\quad+\quad+\quad-\quad-\quad+\quad-\quad+\quad+\quad+\quad-\quad-\quad+\quad-\quad+\quad-\quad+\quad-\quad-\quad+\quad+\quad-\quad-\quad+\quad+\quad-\quad+\quad+\quad+\quad+\quad-\quad+\quad-\quad+\quad+\quad+\quad-\quad+\quad+\quad+\quad-\quad-\quad+\quad+\quad$ }

SEQ B (PAPR=4.4229): { ++++++ - + - - + + - + - - + - - + - + - - - - - + + - + - - - - + + + - - + - - - - -
- }

Pair #2 PAPR: 3.5241 (5.4705 dB)

SEQ A (PAPR=3.5241): {
 -----++--+-++++-+-+--+--++--++-++++-+-++++-+++-+
 + }

SEQ B (PAPR=3.1691): { ++++++-----+-++-----+-++-----+-++-----+-
- }

結論： 對於"已知部份A序列" 求中等長度 $L=50$ PCP 問題，也花費三天時間才建構出第二組，這方法也會隨長度增加變得難以執行

CHALLENGE

Taking a sequence of length 74 as an example, the number of combinations in the search space is approximately 10^{21} , which is computationally impractical and would require an extremely large amount of time to complete.

SOLUTION

In "Some new periodic Golay pairs," the authors apply concepts from algebraic group theory to the search for PCPs, and then verify the assumptions made. The results prove the authors' assumptions to be true, and this method significantly reduces the number of combinations that need to be searched.

STRATEGY FOUNDATION

- Transform the search sequence into an equivalent algebraic problem.
- $PCP \Leftrightarrow SDS$: Finding a pair of PCP sequences (A, B) is equivalent to finding a pair of SDS sequences

(X, Y).

➤ Definition of (X , Y) : X=The set of indices of all values of -1 in sequence A

Y=The set of indices of all values of -1 in sequence B

➤ SDS parameter : Defined by a set of parameters ($v ; r ; s ; \lambda$)

SDS

- v : Length of the sequence
- r, s : The size of sets X and Y is equal to the total number of -1s in sequences a and b .
- By theorem, if (a, b) is a PCP then $A^2 + B^2 = 2L$ where

$$A = \sum_{i=0}^{L-1} a_i \text{ and } B = \sum_{i=0}^{L-1} b_i, \text{ then we can obtain that } (L - 2r)^2 + (L - 2s)^2 = 2L$$

$$pf. A = (L - r) \times (1) - (r) \times (-1) = L - 2r$$

$$B = (L - s) \times (1) - (s) \times (-1) = L - 2s$$

- λ : Number of times common differences occur

$\lambda = N_X(u) + N_Y(u)$ where N_X is the number of pairs (α, β) in X which (α, β) is the index of -1 in the set X and u is the difference between (α, β) .

SDS

- λ : Number of times common differences occur
 - $\lambda = N_X(u) + N_Y(u)$ where N_X is the number of pairs (α, β) in X which (α, β) is the index of -1 in the set X and u is the difference between (α, β) .
 - Regardless of the value of u , λ is a constant.

SUBGROUP H

- Target : We assume that the solution (X, Y) we are looking for is not random, but has a "structural regularity", then introduce algebraic symmetry.
- This "pattern" is the "algebraic tool" we use to simplify problems.
- How to find H ?
 1. Locked range : We find H in Z_v^* which is the multiplication group of Z_v .
 2. Range size $\phi(v)$: Use Euler's totient function to find the cardinality of Z_v^* .
 - $\phi(n) = n \left(1 - \frac{1}{p_1}\right) \left(1 - \frac{1}{p_2}\right) \dots$ where p_i is the prime factors of n
 - E.g. $\phi(74) = 74 \left(1 - \frac{1}{2}\right) \left(1 - \frac{1}{37}\right) = 36$

SUBGROUP H

- How to find H ?

3. List the possible order of H : By Lagrange's theorem the size of the subgroup of H should be the factor of $\phi(v)$.

4. Structure of H : We choose an order of H and find a generator g of that order.

- H is a cyclic subgroup generating by H
- E.g. $H = \langle 47 \rangle = \{1, 47, 63\}$

ORBIT

- Target : From "selecting elements" to "selecting tracks" in order to simplify search assumptions
- Orbit
 1. The "symmetric element" produced by H acting on Z_v .
 2. Definition. $O_j = \{h \cdot j \pmod v \mid h \in H\}$
 - Trivial Orbit : $|O_j| = 1$ which j satisfies $h \cdot j = j \pmod v$
 - Nontrivial Orbit : $|O_j| > 1$

INDEX SET J

- j : the element in Z_v use to generate the orbit $H \cdot j$
- J : an index set which records all the orbits j that make up X
- How to find J : Through large-scale computer search, the algorithm generates millions of candidate "track unions," then compares their "difference distributions" to find the set (X, Y) that exactly satisfies the λ condition.

CONSTRUCTION(L=74)

- Batch A = 5000 :
- 一次存5000個A序列的PACF，搜索全部的B序列，避免過多重複計算的PACF

```
// ----- 參數 -----  
const int KA = 12;           // A: 選 12 個 orbit 為 -1  
const int KB = 10;          // B: 選 10 個 orbit 為 -1  
#define BATCH_A 5000        // 一批最多 5000 個 A  
const unsigned long long TOTAL_A_COMBOS = 2704156ULL; // C(24,12)
```

Sequence	# of +1	# of -1
A	38	36
B	43	31

目標：A有36個-1、B有31個-1

A (orbit order = 3)：總共選取12個 3 elements orbits
A序列組合數共C24取12個 = 2704156

B (orbit order = 3)：選取10個 3 elements orbits + 1 nontrivial
B序列組合數共C(24,10)*C(2,1) = 3922512

CONSTRUCTION(L=74)

Step 1. 計算 ϕ

```
int phi(int v) {
    int count = 0;
    for (int i = 1; i < v; i++) {
        if (gcd(v, i) == 1) {
            count++;
        }
    }
    return count;
}
```

Step 2. 計算 order

```
int get_order(int g, int v) {
    if (g <= 0 || gcd(g, v) != 1) {
        return -1; // 不在  $\mathbb{Z}_v^*$  中
    }

    long long p = g;
    for (int k = 1; k < v; k++) {
        if (p == 1) {
            return k; // 階是 k
        }
        p = (p * g) % v;
    }
    return -1;
}
```

CONSTRUCTION(L=74)

Step 3. 找出n的因數

```
void find_divisors(int n, int divisors[], int* count) {
    *count = 0;
    for (int i = 1; i <= n; i++) {
        if (n % i == 0) {
            divisors[(*count)++] = i;
        }
    }
}
```

Step 4. 尋找軌道

```
void find_orbits(int v, int H_elements[], int H_size) {
    printf(" Orbits of Z_%d under H:\n", v);

    int visited[MAX_V];
    memset(visited, 0, sizeof(visited)); // 初始化為 0

    int total_trivial = 0;
    int total_nontrivial = 0;

    // 遍歷 Z_v 中的每一個元素 j
    //
    for (int j = 0; j < v; j++) {
        if (!visited[j]) {
            int orbit[MAX_V];
            int orbit_size = 0;

            // 計算 H * j
            for (int k = 0; k < H_size; k++) {
                int h = H_elements[k];
                int orbit_element = (1LL * h * j) % v;

                int found = 0;
                for (int m = 0; m < orbit_size; m++) {
                    if (orbit[m] == orbit_element) {
                        found = 1;
                        break;
                    }
                }

                if (!found) {
                    orbit[orbit_size++] = orbit_element;
                    visited[orbit_element] = 1;
                }
            }
        }
    }
}
```

CONSTRUCTION(L=74)

Step 5. 找出Orbit

```
// 3a: 尋找一個階為 d 的生成元 g
int g = -1;
for (int j = 1; j < v; j++) {
    if (get_order(j, v) == d) {
        g = j;
        break; // 找到一個就夠了
    }
}

if (g == -1) {
    printf(" 未能在  $\mathbb{Z}_{v^*}$  中找到階為 %d 的生成元。\\n", v, d);
    continue;
}

// 3b: 構造 H 並列出元素
printf(" 3b: 找到一個生成元 g = %d\\n", g);
printf("      對應的循環子群 H = { ");

int H_elements[MAX_DIVISORS];
for (int k = 0; k < d; k++) {
    H_elements[k] = (int)power(g, k, v);
    printf("%d ", H_elements[k]);
}
printf("}\\n\\n");

// 3c: 找出 Trivial 和 Nontrivial 軌道
printf(" 3c: 找出 H 作用在  $\mathbb{Z}_{v^*}$  上的軌道:\\n", v);
find_orbits(v, H_elements, d);
```

ORBIT(L=74)

請輸入序列長度 (v): 74

=====

步驟 0: 尋找 (r, s) 候選配對 (基於 Theorem 1)

$$(L-2r)^2 + (L-2s)^2 = 2L \quad (L=74)$$

$$\text{尋找 } (A')^2 + (B')^2 = 148$$

找到一組解: $A' = 2, B' = 12 \Rightarrow (r, s) = (36, 31)$
找到一組解: $A' = 2, B' = -12 \Rightarrow (r, s) = (36, 43)$
找到一組解: $A' = -2, B' = 12 \Rightarrow (r, s) = (38, 31)$
找到一組解: $A' = -2, B' = -12 \Rightarrow (r, s) = (38, 43)$
找到一組解: $A' = 12, B' = 2 \Rightarrow (r, s) = (31, 36)$
找到一組解: $A' = 12, B' = -2 \Rightarrow (r, s) = (31, 38)$
找到一組解: $A' = -12, B' = 2 \Rightarrow (r, s) = (43, 36)$
找到一組解: $A' = -12, B' = -2 \Rightarrow (r, s) = (43, 38)$

=====

步驟 1: 乘法群 $\mathbb{Z}_{74}^*$ 的大小 ($\phi(74)$) = 36

步驟 2: $\mathbb{Z}_{74}^*$ 中可能的子群階 (Order) ($\phi(36)$ 的因數) 有 9 個:
{ 1 2 3 4 6 9 12 18 36 }

=====

步驟 3: 分析 Order d = 3

3b: 找到一個生成元 $g = 47$

對應的循環子群 $H = \{ 1 \ 47 \ 63 \}$

3c: 找出 H 作用在 $\mathbb{Z}_{74}$ 上的軌道:

Orbits of $\mathbb{Z}_{74}$ under H:

{ 0 } (Trivial)
{ 1 47 63 } (Nontrivial)
{ 2 20 52 } (Nontrivial)
{ 3 67 41 } (Nontrivial)
{ 4 40 30 } (Nontrivial)
{ 5 13 19 } (Nontrivial)
{ 6 60 8 } (Nontrivial)
{ 7 33 71 } (Nontrivial)
{ 9 53 49 } (Nontrivial)
{ 10 26 38 } (Nontrivial)
{ 11 73 27 } (Nontrivial)
{ 12 46 16 } (Nontrivial)
{ 14 66 68 } (Nontrivial)
{ 15 39 57 } (Nontrivial)
{ 17 59 35 } (Nontrivial)
{ 18 32 24 } (Nontrivial)
{ 21 25 65 } (Nontrivial)
{ 22 72 54 } (Nontrivial)
{ 23 45 43 } (Nontrivial)
{ 28 58 62 } (Nontrivial)
{ 29 31 51 } (Nontrivial)
{ 34 44 70 } (Nontrivial)
{ 36 64 48 } (Nontrivial)
{ 37 } (Trivial)
{ 42 50 56 } (Nontrivial)
{ 55 69 61 } (Nontrivial)

Total Trivial Orbits: 2, Total Nontrivial Orbits: 24

=====

步驟 3: 分析 Order d = 4

3b: 找到一個生成元 $g = 31$

對應的循環子群 $H = \{ 1 \ 31 \ 73 \ 43 \}$

3c: 找出 H 作用在 $\mathbb{Z}_{74}$ 上的軌道:

Orbits of $\mathbb{Z}_{74}$ under H:

{ 0 } (Trivial)
{ 1 31 73 43 } (Nontrivial)
{ 2 62 72 12 } (Nontrivial)
{ 3 19 71 55 } (Nontrivial)
{ 4 50 70 24 } (Nontrivial)
{ 5 7 69 67 } (Nontrivial)
{ 6 38 68 36 } (Nontrivial)
{ 8 26 66 48 } (Nontrivial)
{ 9 57 65 17 } (Nontrivial)
{ 10 14 64 60 } (Nontrivial)
{ 11 45 63 29 } (Nontrivial)
{ 13 33 61 41 } (Nontrivial)
{ 15 21 59 53 } (Nontrivial)
{ 16 52 58 22 } (Nontrivial)
{ 18 40 56 34 } (Nontrivial)
{ 20 28 54 46 } (Nontrivial)
{ 23 47 51 27 } (Nontrivial)
{ 25 35 49 39 } (Nontrivial)
{ 30 42 44 32 } (Nontrivial)
{ 37 } (Trivial)

Total Trivial Orbits: 2, Total Nontrivial Orbits: 18

ORBIT(L=74)

- **Total orbits :**
- Trivial orbits: {0}, {37} 共2個orbits
- Nontrivial orbits: below 共24個orbits

```
// ----- 24 個 orbit (每個大小 3)，不含 0 與 37 -----  
#define NUM_ORBITS 24  
#define ORBIT_SIZE 3  
  
int orbits[NUM_ORBITS][ORBIT_SIZE] = {  
    {1, 47, 63},{3, 41, 67},{2, 20, 52},  
    {4, 30, 40},{5, 13, 19},{6, 8, 60},  
    {7, 33, 71},{9, 49, 53},{10, 26, 38},  
    {11, 27, 73},{12, 16, 46},{14, 66, 68},  
    {15, 39, 57},{17, 35, 59},{18, 24, 32},  
    {21, 25, 65},{22, 54, 72},{23, 43, 45},  
    {28, 58, 62},{29, 31, 51},{34, 44, 70},  
    {36, 48, 64},{42, 50, 56},{55, 61, 69}  
};
```

CONSTRUCTION

- Calculation of PACF :
- 計算PACF，利用判斷式取代乘法運算

```
int periodic_autocorrelation(int *seq, int shift) {  
    int sum = 0;  
    for (int i = 0; i < L; i++) {  
        int j = i + shift;  
        if (j >= L) j -= L;  
        sum += (seq[i] == seq[j]) ? 1 : -1;  
    }  
    return sum;  
}
```

CONSTRUCTION

- Calculation of next combination :
- 搜索下一個組合

```
int next_combination(int *idx, int n, int k) {  
    int i = k - 1;  
    while (i >= 0 && idx[i] == n - k + i) {  
        i--;  
    }  
    if (i < 0) return 0; // 沒有下一個組合  
  
    idx[i]++;  
    for (int j = i + 1; j < k; j++) {  
        idx[j] = idx[j - 1] + 1;  
    }  
    return 1;  
}
```

目標：計算下一個組合 NUM_ORBITS=24 KA=12 KB=10

step 1:將所有orbit order = 3進行編號0~23

idx[12] = 存取被選取的編號(以A序列為例)

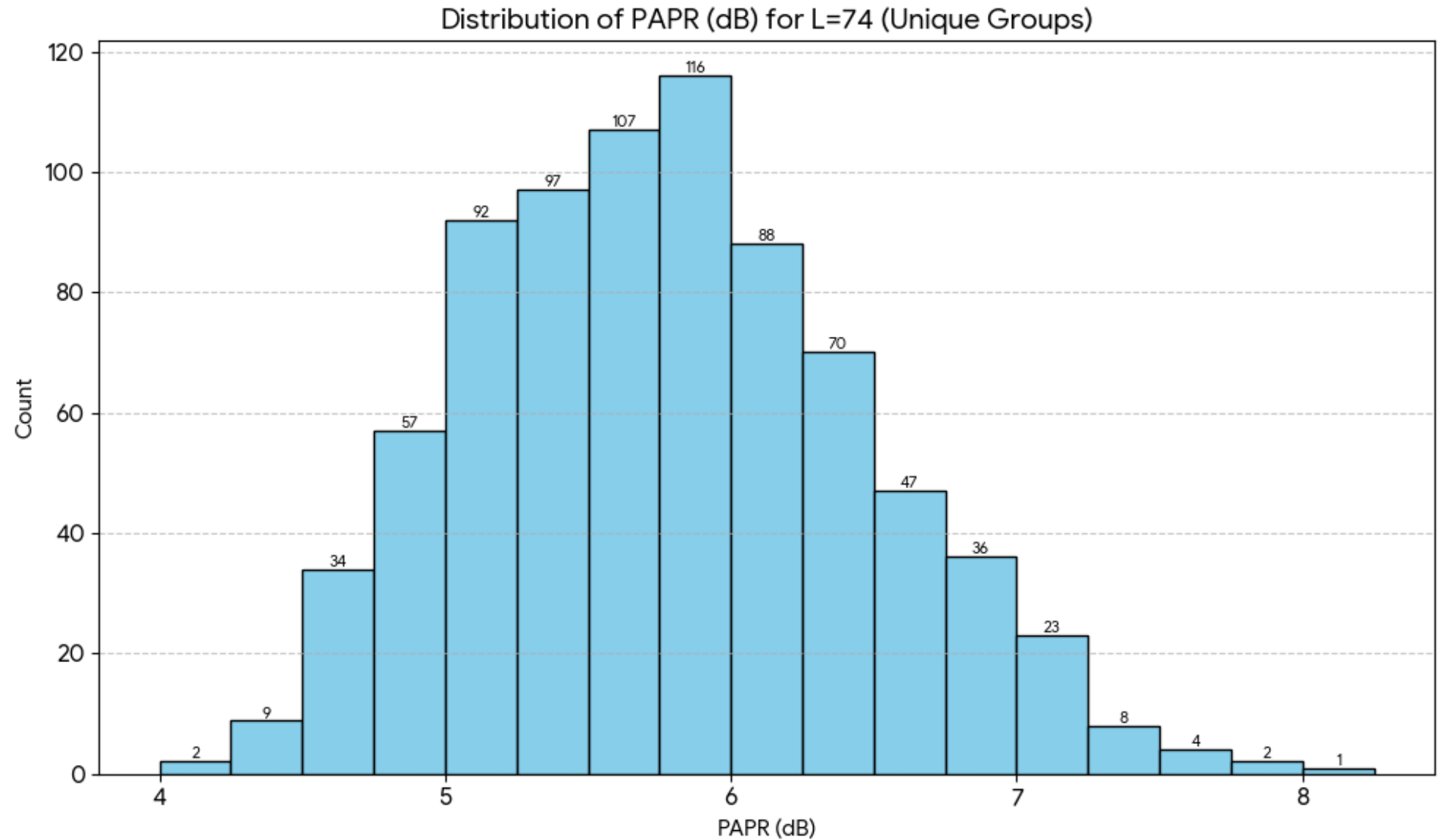
step 2:逐步生成下一個組合

第一組: { 0, 1, 2,..., 9, 10, 11}

第二組: { 0, 1, 2,..., 9, 10, 12}

最後一組:{ 12, 13, 14, ... , 21, 22, 23}

UNIQUE PCP FOR L=74 (共793組)



Min PAPR: 2.5419 (4.0516 dB)

- SEQ A (PAPR=2.5419):

$$\{ ++- --+ - -- -- ++ + - + + + + + - + + - + + - + - + - + + - - + + + - + - - + - - - + + + - + + - - + + - + - - - + + + - + + - + + - + - - - + + \}$$

- SEQ B (PAPR=2.4539):

$$\{ - + + + - + - + - - + + - + + - - - + + + + + - + + + + - + - + + + + - - + + - - + + - + - - + - - + + + - + - + - - - - + + \}$$

ORBIT(L=82)

- Total orbits:
- Trivial orbits: $\{0\}$, $\{41\}$ 共2個orbits
- Nontrivial orbits: below 共16個orbits

```
// 16 orbits of size 5
#define NUM_ORBITS5 16
int orbits5[NUM_ORBITS5][5] = {
    {1, 37, 57, 59, 51},
    {2, 74, 32, 36, 20},
    {3, 29, 7, 13, 71},
    {4, 66, 64, 72, 40},
    {5, 21, 39, 9, 49},
    {6, 58, 14, 60, 26},
    {8, 50, 46, 62, 80},
    {10, 42, 78, 16, 18},
    {11, 79, 53, 75, 69},
    {12, 28, 34, 52, 38},
    {15, 63, 35, 27, 65},
    {17, 55, 67, 19, 47},
    {22, 24, 76, 68, 56},
    {23, 31, 81, 45, 25},
    {30, 70, 48, 44, 54},
    {33, 73, 77, 61, 43}
};
```

```
// 2 trivial orbits of size 1
#define NUM_ORBITS1 2
int orbits1[NUM_ORBITS1] = {
    0, 41
};
```

CONSTRUCTION(L=82)

- **TOTAL_A_COMBOS 11440:**

➤ 一次存5000個A序列的PACF，搜索全部的B序列，避免過多重複計算的PACF

```
int orbits5[NUM_ORBITS5][5] = {  
    {1, 37, 57, 59, 51}, {2, 74, 32, 36, 20}, {3, 29, 7, 13, 71}, {4, 66, 64, 72, 40},  
    {5, 21, 39, 9, 49}, {6, 58, 14, 60, 26}, {8, 50, 46, 62, 80}, {10, 42, 78, 16, 18},  
    {11, 79, 53, 75, 69}, {12, 28, 34, 52, 38}, {15, 63, 35, 27, 65}, {17, 55, 67, 19, 47},  
    {22, 24, 76, 68, 56}, {23, 31, 81, 45, 25}, {30, 70, 48, 44, 54}, {33, 73, 77, 61, 43}  
};
```

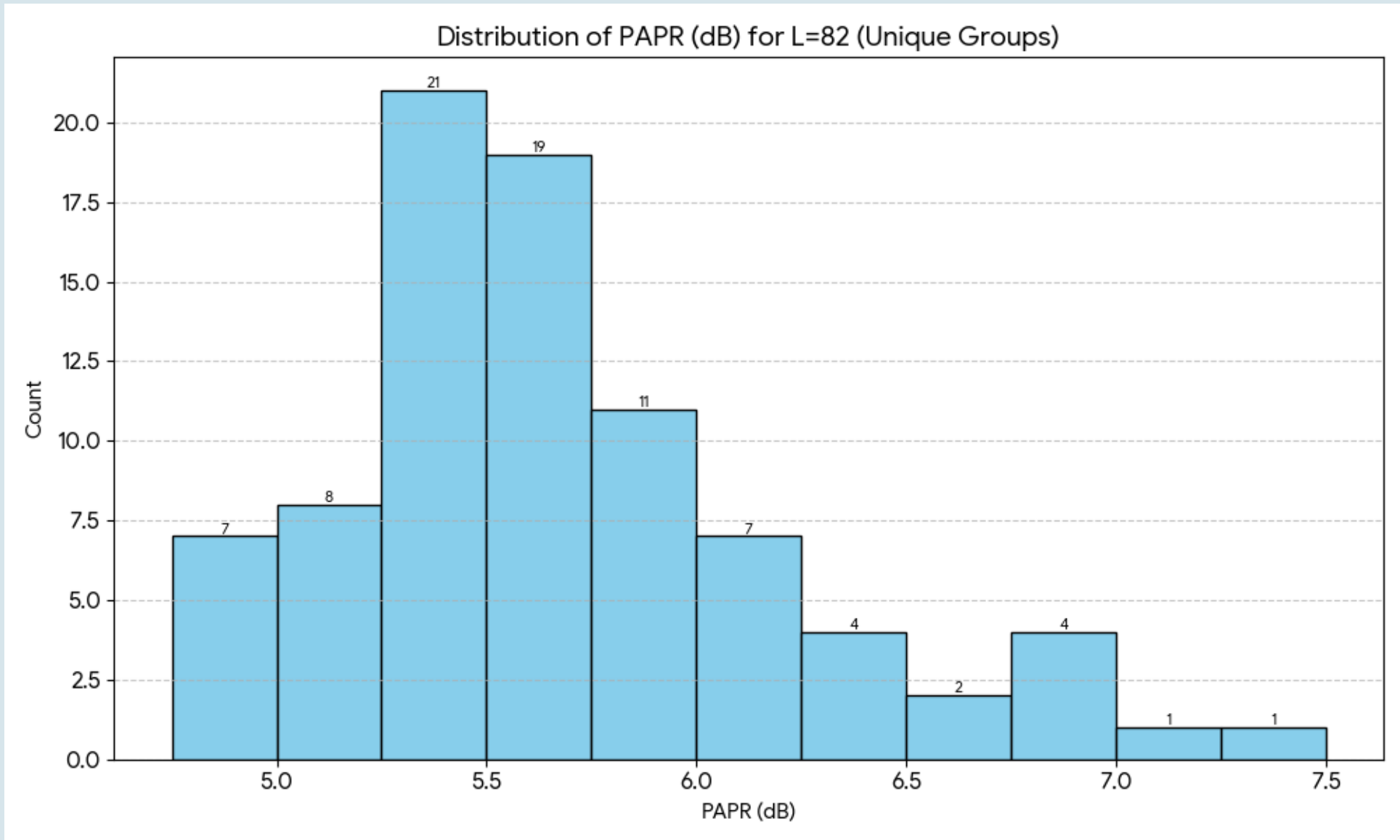
Sequence	# of +1	# of -1
A	37	45
B	46	36

目標：A有45個-1、B有36個-1

A (orbit order = 5)：總共選取9個 5's element orbits
A序列組合數共 $C(16,9)$ = 11440

B (orbit order = 5)：選取7個 5's elements orbits + 1 nontrivial
B序列組合數共 $C(16,7)*C(2,1)$ = 22880

UNIQUE PCP FOR $L=82$ (共85組)



Min PAPR: 3.0201 (4.8002 dB)

- SEQ A (PAPR= 3.0201):

$$\{ \begin{array}{l} + - - - + - - - - - + + - - - + + - + - - - - + - + - + - - + + - + - + - - - - + + + + + + - - + - - - - + + - - - - \\ + \end{array} \}$$

- SEQ B (PAPR= 2.8949):

$$\{ ++-+-+--++--+-+--+---+----++++-+-++++----+-+--+---+++--+--+--+---+++--+--+--+---+ \}$$

序列壓縮法 (EX. $L=90$)

為何不使用 SDS 建構?

SDS 利用循環群結構進行代數構造，通常依賴特定的存在性定理。對於 $L=90$ 這種長度，尚未找到有效的代數構造公式，理論推導遇瓶頸。

序列壓縮搜尋法 (本研究採用)

- **原理：** 將長序列摺疊成短序列，搜尋短序列後再還原。
- **優勢：** $L=90$ 擁有豐富的因數 (2, 3, 5)。這非常適合**多層級壓縮**，能將大問題拆解為多個小問題

壓縮與有序生成

※ 序列壓縮 (Sequence Compression)

將長度 L 的序列壓縮為長度 d 的短序列 ($d = \frac{L}{M}$)

優點：大幅縮減搜尋長度，將問題簡化。

代價：壓縮後的序列元素不再只是 $+1/-1$ ，而是整數的和。

策略：先搜尋短的壓縮序列，再「解壓縮」回原長度。

↓ 有序生成 (Orderly Generation)

一種「無同構」的生成演算法。

原理：在生成過程中，僅保留「字典序最小」的代表序列。

效果：直接剔除旋轉、反轉等對稱變體，避免重複計算。

這對於 $L=90$ 的巨大空間是必要的剪枝手段。

解壓縮是如何運作的? (範例)

假設：我們有一個長度為 2 的壓縮序列 $[2, 0]$

目標：解壓縮為長度 4 (壓縮因子 $m=2$)

步驟 1 (解析數值 2):

// 數值 2 來自兩個位置的和 ($\text{pos } 0 + \text{pos } 2$)

可能組合：只有 $1 + 1 = 2$ 一種可能。

(確定原序列第 1, 3 位為 $+1$)

步驟 2 (解析數值 0):

// 數值 0 來自兩個位置的和 ($\text{pos } 1 + \text{pos } 3$)

可能組合： $1 + (-1)$ 或 $(-1) + 1$ 。

(產生分支：這就是為什麼解壓縮需要搜尋)

💡 現象描述

我們在較小長度（如 $L=34, 50$ ）的窮盡搜尋中發現了一個模式：

許多有效的PCP對，在經過高度壓縮後（例如壓縮到極短長度），其壓縮序列的週期性自相關函數 (PAF) 值全部為零，通常這意味著壓縮後的序列由大量的 0 組成

啟發式策略： 與其搜尋所有可能的壓縮序列，不如直接鎖定那些「具有全零 PAF 特性」的稀疏序列作為起點（種子）

實際應用：L=90 的多層級路徑

利用定理：d-壓縮的 e-壓縮等同於 de-壓縮，且直接從長度 5 解壓縮到 90 是不可能的，我們採取分步驟：

🌱 **初始種子** (長度 5) 使用種子 $[0,0,0,0,6]$ 與 $[0,0,0,0,12]$ 。
壓縮因子 18

📦 **階段 1 解壓** (長度 15)
解壓縮因子 3

📦 **階段 2 解壓** (長度 45)
解壓縮因子 3

🚩 **最終目標** (長度 90)
解壓縮因子 2

💡 L = 90多層級壓縮&解壓縮實例

初始種子 (長度 5) 使用 $[0,0,0,0,6]$ 與 $[0,0,0,0,12]$ 。

壓縮因子 18

階段 1 解壓 (長度 15)

解壓縮因子 3

Let sequence $x = \{x_0, x_1, x_2, x_3, \dots, x_{89}\}$

compress: $d=18, L=5$

$a = [a_0, a_1, a_2, a_3, a_4]$

$a_0 = x_0 + x_5 + x_{10} + \dots + x_{85} = 0$

$a_1 = x_1 + x_6 + x_{11} + \dots + x_{86} = 0$

$a_2 = x_2 + x_7 + x_{12} + \dots + x_{87} = 0$

$a_3 = x_3 + x_8 + x_{13} + \dots + x_{88} = 0$

$a_4 = x_4 + x_9 + x_{14} + \dots + x_{89} = 6$

$a_0 \sim a_4$ 的可能值為 $-18 \sim +18$, 共19種

decompress: $d=6, L=15$

$b = [b_0, b_1, b_2, b_3, \dots, b_{14}]$

$a_0 = b_0 + b_5 + b_{10} = 0$

$a_1 = b_1 + b_6 + b_{11} = 0$

$a_2 = b_2 + b_7 + b_{12} = 0$

$a_3 = b_3 + b_8 + b_{13} = 0$

$a_4 = b_4 + b_9 + b_{14} = 6$

$b_0 \sim b_{14}$ 的可能值為 $-6 \sim +6$, 共7種

$b_0 \sim b_{14} \in \{-6, -4, -2, 0, 2, 4, 6\}$

可能的 $a_n = 0$ 組法:

$(0,0,0)$
 $(2,-2,0)$
 $(4,-4,0)$
 $(6,-6,0)$
 $(2,4,-6)$
 $(-2,-4,6)$
 $(6,2,-2)$

可能的 $a_n = 6$ 組法:

$(6,0,0)$
 $(2,2,2)$
 $(2,4,0)$
 $(4,4,-2)$
 $(6,4,-4)$
 $(6,6,-6)$
 $(6,2,-2)$

檢查壓縮序列

頻譜抽樣定理

這背後的數學原理是：**時域上的壓縮 (疊加)**，等同於**頻域上的抽樣 (SUBSAMPLING)**。

$$\text{PSD}_{\text{compressed}}(k) = \text{PSD}_{\text{original}}(mk)$$

可知道壓縮序列的PSD會符合1、2狀態

1. 能量恆等式 (Equation)

$$\text{PSD}(A,s) + \text{PSD}(B,s) = 2L$$

由來 (Wiener-Khinchin 定理):

PCP 定義要求自相關函數 (PAF) 之和為 0 (除了零點)。對 PAF 做傅立葉變換，時域的 PAF 變成頻域的 PSD。PAF 在頻域上會變成常數 2L。

2. 能量上限 (Inequality) - 最強篩子

$$\text{PSD}(A,s) \leq 2L$$

證明:

因為 PSD 是絕對值的平方，必定大於等於 0。既然 $\text{PSD}(A) + \text{PSD}(B) = 2L$ 。因為 $\text{PSD}(A)$ 、 $\text{PSD}(B)$ 不可能是負的，所以 $\text{PSD}(A)$ 絕對不能超過 2L。

UNIQUE PCP FOR $L=90$ (共2組)

Pair #1 PAPR: 5.0602 (7.041 dB)

SEQ A (PAPR=4.1498):

$$\{ \begin{array}{l} - - - - + + - + - + - - + + - + + - + - - + + + + - + + + - + + + - - - + + - + + + - - + + - - - - + + + + - - + + + - + - - + + - - + - + + + \\ - - + + - + - + - - + + - + - - \end{array} \}$$

SEQ B (PAPR=5.0602):

$$\{ \begin{aligned} & - - - + - - + + + + + + + + - - - - + - - + + - - - - + + + + + + + + - - + + - + - + - + + - + - + + + + - + - + + - + + + + - - + + + + - + - + + + - + \\ & + \} \end{aligned}$$

Pair #2 PAPR: 3.8064 (5.805 dB)

SEQ A (PAPR=2.9374):

[illegible]

SEQ B (PAPR=3.8064):

$$\{ \begin{array}{l} - - - + - - - + - + - - - + + + - - + + - - + - + - + - - + + + + - + - - + - + + + + - + + + - - - - + - + + + - - + + - + + + + + + + + + + + + + - - + + - \\ + \end{array} \}$$

ALL RESULT

Metric	L=74	L=82	L=90
Total Pairs	793	85	2
Min PAPR	2.5419 (4.0516 dB)	3.0201 (4.8002 dB)	3.8064 (5.805 dB)

參考文獻

- [1] D. Z. Doković and I. S. Kotsireas, “Some new periodic Golay pairs,” *Numer. Algorithms*, vol. 69, no. 3, pp. 523–530, Sep. 2015.
- [2] T. Lumsden, I. Kotsireas, and C. Bright, “New results on periodic Golay pairs,” *Math. Comput.*, Apr. 2025.

END